Informe Técnico — Fase 2: Respaldo y Restauración

Proyecto: `775_Proyecto_2_2S2026.md`.

Ejecución documentada: **2026-10-16**.

1. Metodología

Se trabajó sobre la base de datos de Olimpiadas de Fase 1, con el mismo modelo entidad-relación (10 tablas, ver sección 2). En vez de repetir el ETL de las 7 fuentes originales, se generaron subconjuntos a partir de los CSV ya procesados en `Fase1/etl-csvs/` (ver `carga/generar_subconjuntos.py`), uno por cada ronda pedida por el enunciado:

Ronda	Carga 1	Carga 2	Carga 3
Por año	París 2024	Tokio 2020	Río 2016
Por deporte	Atletismo 100m mujeres — París 2024	ídem — Tokio 2020	ídem — Río 2016
Por deportista	Usain Bolt — Atenas 2004	ídem — Pekín 2008	ídem — Londres 2012

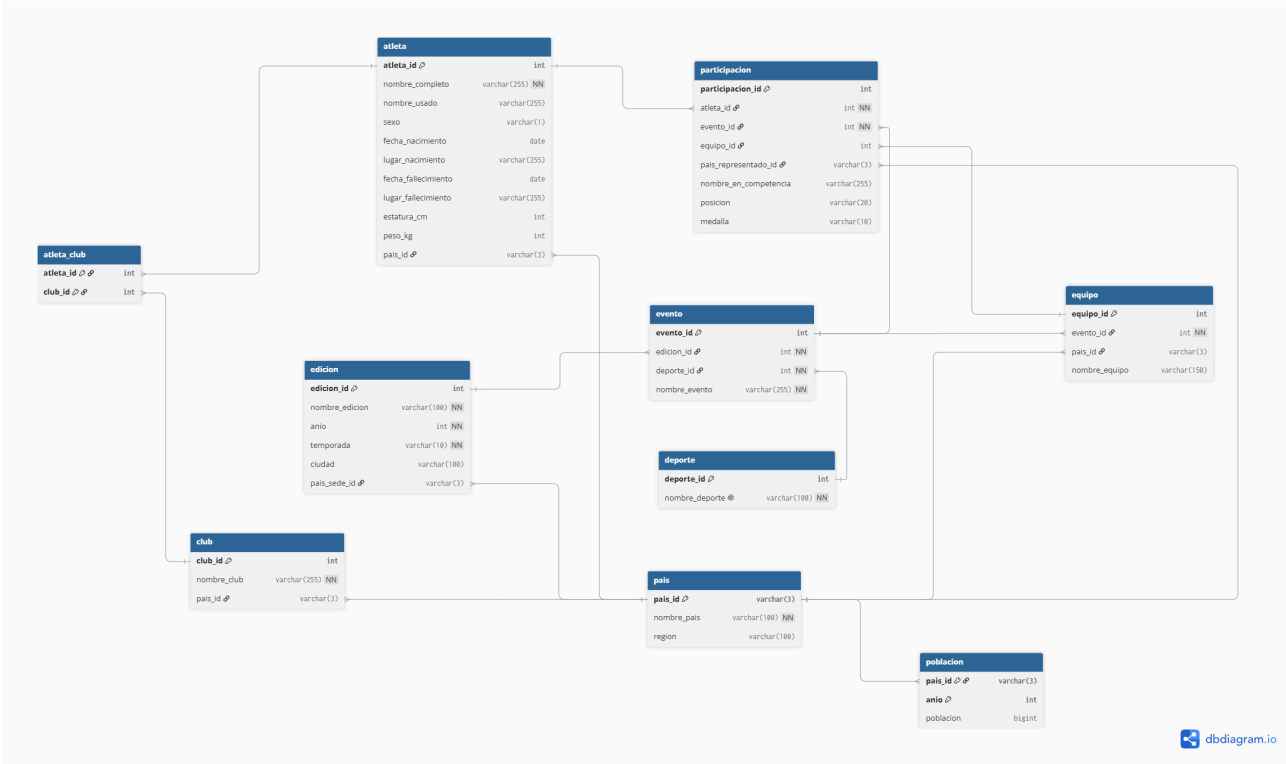
Cada paso inserta **solo las filas nuevas** respecto al paso anterior (carga delta vía `COPY`), para que el crecimiento de la base de datos entre cargas sea real y el respaldo incremental tenga algo que capturar. Para cada ronda, en una instancia nueva: carga 1 → respaldo completo → carga 2 → respaldo incremental → carga 3 → respaldo incremental → se elimina la base de datos por completo → se restaura solo el full (con tiempo) → se restaura full + incrementales (con tiempo) → se valida integridad.

2. Modelo entidad-relación utilizado

Igual al de Fase 1 (reutilizado sin cambios): `pais`, `poblacion`, `atleta`, `edicion`, `deporte`, `evento`, `equipo`, `club`, `atleta_club`,

`participacion`. Ver [`../../Fase1/informe.md`](../../Fase1/informe.md)

sección 4 para el diagrama y [`../../Fase1/img/ER.png`](../../Fase1/img/ER.png).



El generador de Fase 2 recorre las relaciones del subconjunto y produce deltas para `pais`, `poblacion`, `edicion`, `deporte`, `evento`, `atleta`, `club`, `atleta_club` y `participacion`. En Fase 1, `equipo` se construía después de importar los CSV mediante `etl/derivar_equipos.sql`; por eso el archivo `etl-csvs/equipo.csv` no contiene el resultado derivado. Para mantener el mismo comportamiento, `cargar_paso.ps1` ejecuta [`sql/derivar_equipos.sql`](sql/derivar_equipos.sql) después de cada delta, reconstruye los equipos sobre todas las participaciones acumuladas y vuelve a enlazar `participacion.equipo_id` antes de validar y respaldar. En el estado final por año se obtuvieron 5,108 equipos y 28,647 participaciones enlazadas; los escenarios de 100 metros y un solo deportista conservan cero equipos de forma correcta porque sus subconjuntos no contienen grupos de dos o más atletas del mismo país en un evento colectivo.

3. Plan de respaldo

Herramienta: `pg_basebackup` / `pg_combinebackup` (nativas de PostgreSQL

17, físicas, no dumps). Se descartó `pg_dump` porque el enunciado pide explícitamente una herramienta de respaldo físico.

Por qué PostgreSQL 17: es la primera versión con respaldo incremental nativo (`pg_basebackup --incremental`). Versiones anteriores (como el Postgres 16 usado en Fase 1) solo permiten "incremental" vía archivado continuo de WAL, que es más difícil de operar y de medir por separado.

Configuración: `summarize_wal = on` en el servidor (ver `docker-compose.yml`), que habilita un proceso en segundo plano que resume los cambios de WAL en archivos de resumen — `pg_basebackup --incremental` los usa para identificar qué bloques cambiaron desde el respaldo de referencia.

Estrategia elegida: incremental **encadenado** (cada incremental usa como base el `backup_manifest` del respaldo inmediatamente anterior, no siempre el del full) — ver `backup/respaldo_incremental.ps1` . Se prefirió sobre diferencial porque, en teoría, cada incremental solo contiene los bloques modificados desde el respaldo *inmediato* anterior (más chico cuanto más seguido se respalda), mientras que un diferencial siempre compara contra el full y crece con cada carga. No se implementó la variante diferencial para comparar ambas empíricamente — queda como trabajo futuro; lo que sí se midió (sección 2 de `analisis_comparativo.md`) es que los incrementales pesan aproximadamente 19-41% menos que el full.

Almacenamiento: los respaldos se escriben en `./backups` (carpeta del host, montada en el contenedor), no en el volumen de datos de Postgres — así sobreviven a un `docker compose down -v` (ver sección 8.3 del enunciado, "almacenamiento persistente").

4. Especificaciones técnicas del servidor

- Motor: PostgreSQL 17.11 (imagen `postgres:17-alpine`), vía Docker Desktop en Windows.

- Contenedor principal: `fase2-pg`, puerto 5433 (host) → 5432.
- Volumen de datos: volumen Docker nombrado `fase2_pgdata` (persistente).
- Backups: bind-mount `./backups` del host.
- Restauración de prueba: contenedores efímeros (`fase2-prueba`, puerto 5544) con `postgres:17-alpine`, sin recursos dedicados adicionales a los límites por defecto de Docker Desktop.

5. Fragmentación de las tablas

La metodología (sección 6, Fase 2 del enunciado) pide obtener el nivel de fragmentación de las tablas al final de cada tipo de carga. Se midió con `pg_stat_user_tables` (tuplas vivas vs. muertas) y la extensión `pgstattuple` (detalle a nivel de bytes) — ver sql/verificar_fragmentacion.sql:

```
docker cp sql\verificar_fragmentacion.sql fase2-pg:/verificar_fragmentacion.sql
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -f /verificar_fragmentacion.sql
```

Las inserciones se realizan con `COPY`, pero después de cada paso la derivación idempotente actualiza `participacion.equipo_id`, elimina los equipos anteriores y los reconstruye. Por tanto, `pg_stat_user_tables` puede mostrar tuplas muertas en `participacion` y `equipo` hasta que `autovacuum` las limpie; esto es comportamiento MVCC esperado, no pérdida de integridad. La medición reproducible y el detalle con `pgstattuple` se obtienen con el comando anterior y se contrastan con la validación de cero filas huérfanas. En la medición final, las estadísticas estimaron 46,526 tuplas muertas en `participacion` (38.12%) y 3,367 en `equipo` (39.73%); ambas tablas ya registraban una ejecución de `autovacuum`. La inspección física exacta de `participacion` mediante `pgstattuple`, realizada inmediatamente después, encontró 0 tuplas muertas y 35.85% de espacio libre, mostrando que la estimación estadística todavía no se había actualizado después de la limpieza.

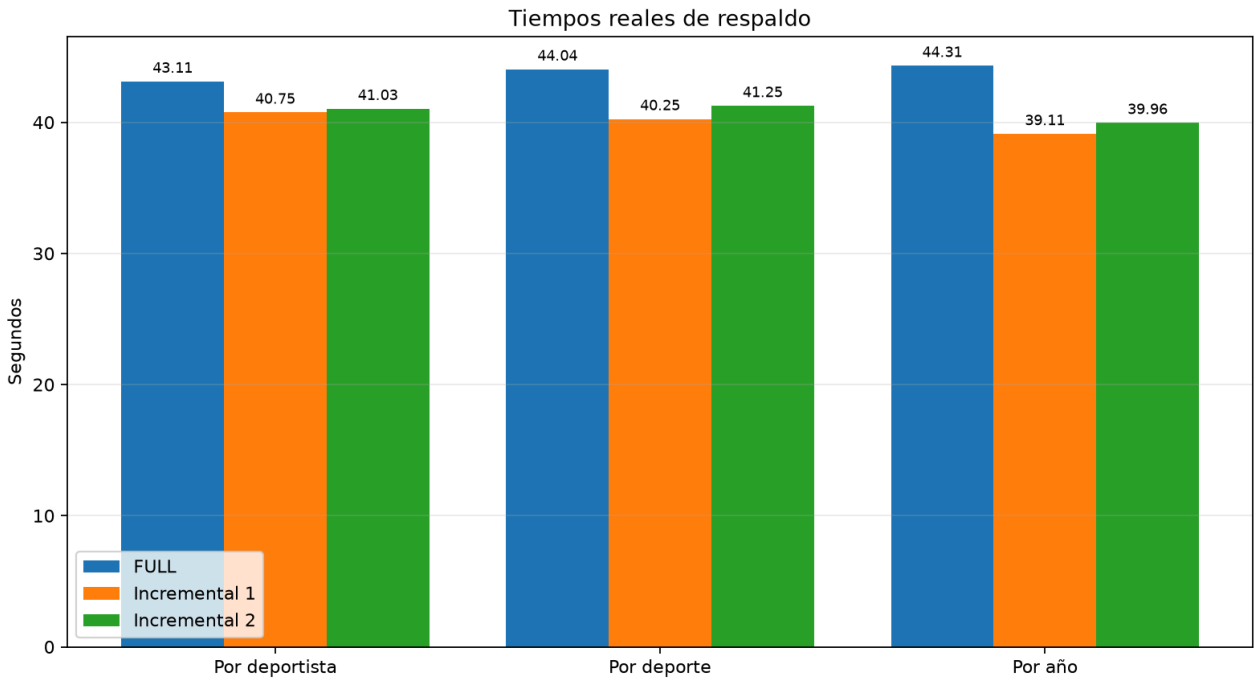
Fragmentación medida después de derivar equipos

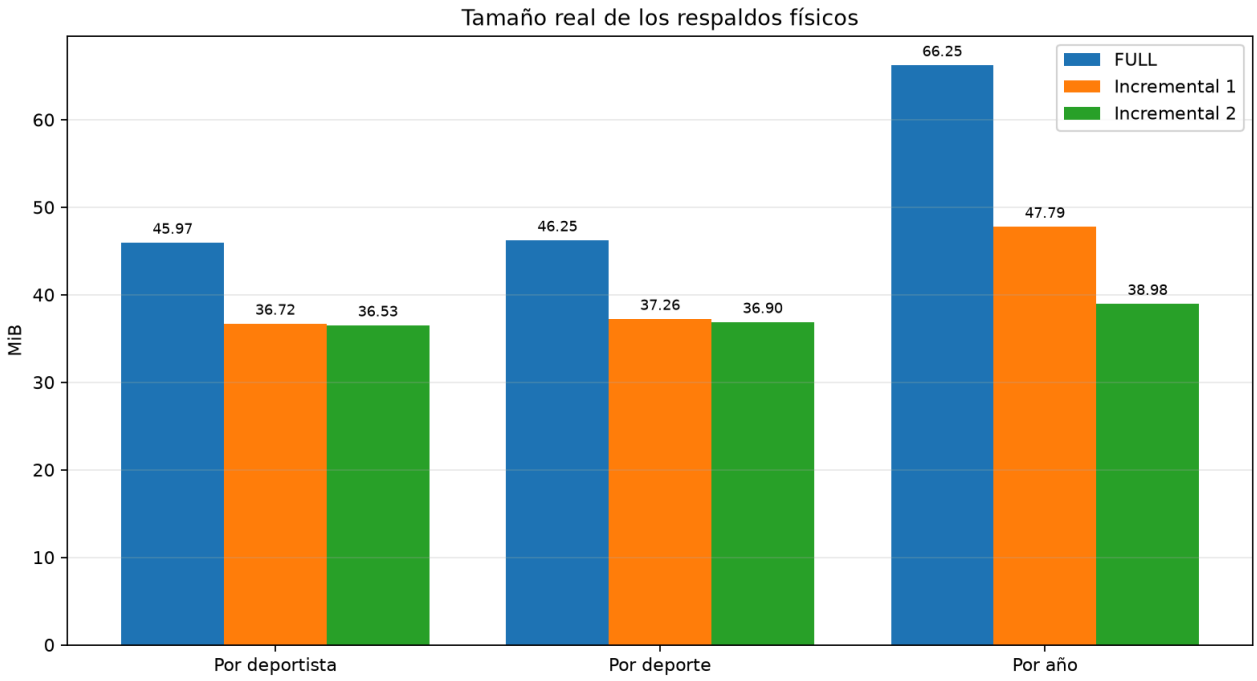
```
tabla           | vivas    | muertas  | estimación %
----------------+----------+----------+-------------
participacion   |   75,533 |   46,526 |       38.12
atleta          |   40,754 |        0 |        0.00
atleta_club     |   15,202 |        0 |        0.00
club            |    9,384 |        0 |        0.00
poblacion       |    8,896 |        0 |        0.00
equipo          |    5,108 |    3,367 |       39.73
evento          |    1,676 |        0 |        0.00
pais            |      228 |        0 |        0.00
deporte         |       85 |        0 |        0.00
edicion         |        3 |        0 |        0.00

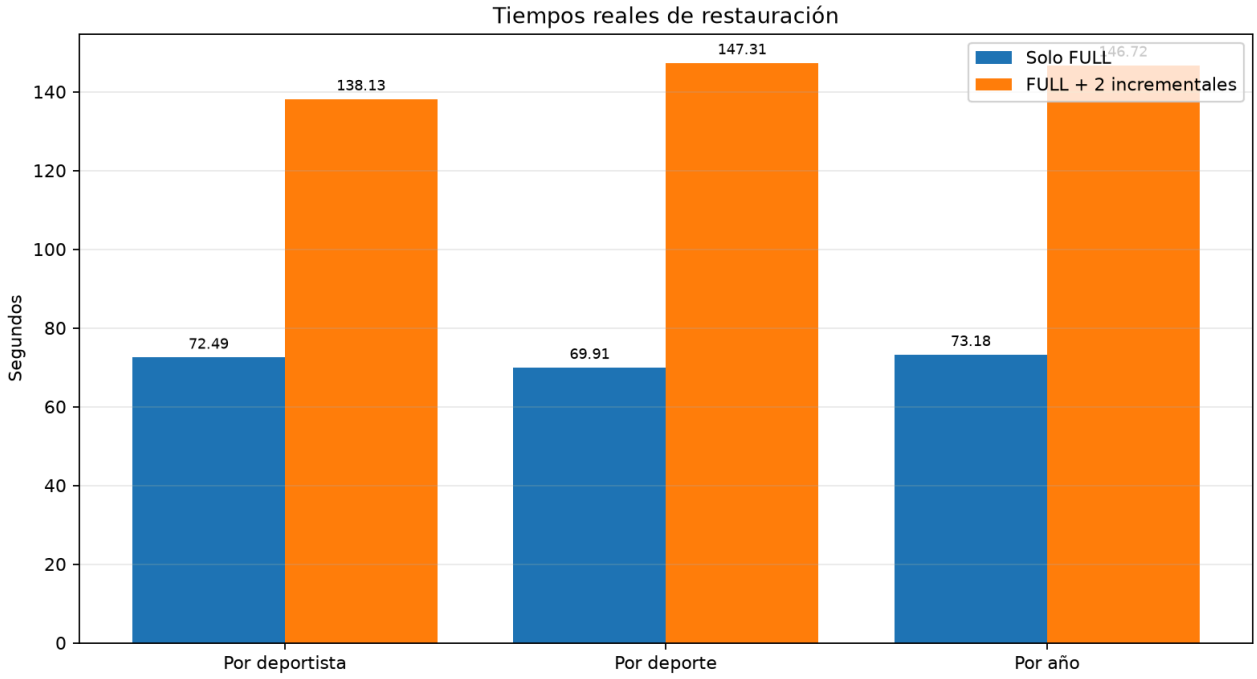
pgstattuple(participacion):
tuplas muertas físicas = 0
espacio libre reutilizable = 35.85%
Integridad referencial = 0 filas huérfanas
```

6. Resultados y análisis comparativo

Ver `analisis_comparativo.md` (tiempos, tamaños, hallazgos y recomendación) y `evidencia.md` (capturas con fecha de cada paso).







7. Conclusiones

1. El respaldo físico incremental de Postgres 17 funciona correctamente sin herramientas de terceros: se validó end-to-end en 3 rondas distintas (24 operaciones de carga/respaldo/restauración en total) sin fallos de integridad.

2. A la escala de datos manejada (decenas de miles de filas, decenas de MB), el tiempo de respaldo está dominado por el overhead fijo de

`pg_basebackup` (checkpoint + streaming de WAL), no por el volumen de datos — la ventaja del incremental se ve en el **tamaño** en disco, no en la velocidad de toma del respaldo.

3. La restauración incremental es consistentemente más lenta que la restauración de solo el full (91%-111% más tiempo en las pruebas nuevas), porque `pg_combinebackup` debe reconstruir el directorio completo antes de poder arrancar el servidor.

4. Restaurar solo el full backup es más rápido pero **incompleto** — en las 3 rondas, el `COUNT(*)` tras restaurar solo el full coincidió exactamente con los datos de la carga 1, confirmando que los incrementales son indispensables para una recuperación total.

5. Recomendación: estrategia híbrida full periódico + incrementales frecuentes, priorizando RPO bajo (minimizar pérdida de datos) sobre RTO (tiempo de restauración), dado que el costo de *tomar* un incremental es similar al de un full pero pesa menos en disco.

6. La derivación de equipos reproduce correctamente el paso de Fase 1 y hace que las diez tablas participen en la validación. Como emplea `UPDATE` y `DELETE`, `autovacuum` sigue siendo necesario para recuperar espacio de tuplas muertas; esto no afecta la consistencia de los respaldos físicos.

8. Validación final de entrega

Antes de cerrar la entrega se restauraron nuevamente los tres full y las tres cadenas completas en un contenedor aislado. Los seis conteos coincidieron con los esperados (2/14 por deportista, 91/410 por deporte y 14,891/75,533 por año), y `sql/validar_integridad.sql` confirmó cero filas huérfanas en cada estado final. También se verificó la sintaxis de todos los scripts PowerShell y Python, los nueve manifiestos físicos y la disponibilidad de `fase2-pg` con 75,533 participaciones. La evidencia reproducible está en `logs/validacion_*`.

Estado final de las diez tablas

```
pais           |      228
poblacion      |    8,896
atleta         |   40,754
edicion        |        3
deporte        |       85
evento         |    1,676
equipo         |    5,108
club           |    9,384
atleta_club    |   15,202
participacion  |   75,533

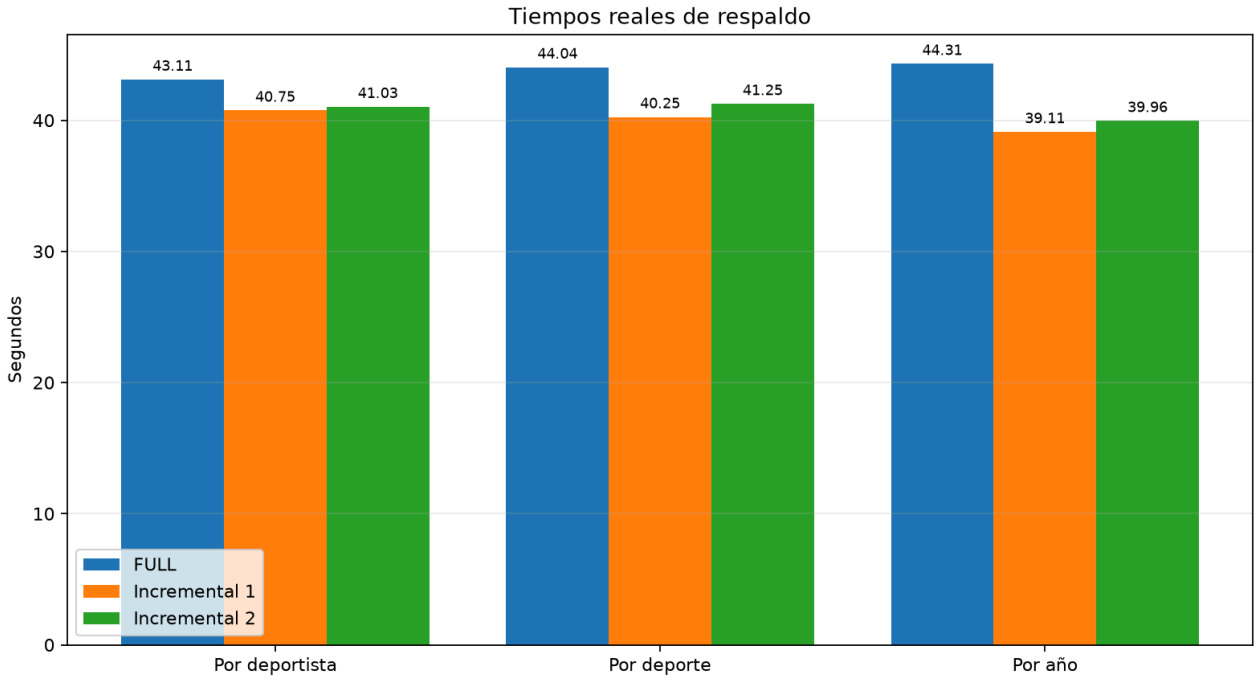
VALIDACION OK: 0 filas huerfanas
```

Análisis Comparativo de Estrategias de Respaldo

Datos reales tomados de `logs/backups.csv` y `logs/restores.csv` durante la ejecución del **2026-10-16** sobre Postgres 17 en Docker Desktop (contenedor `fase2-pg`, 1 CPU/volumen por defecto de Docker Desktop en Windows).

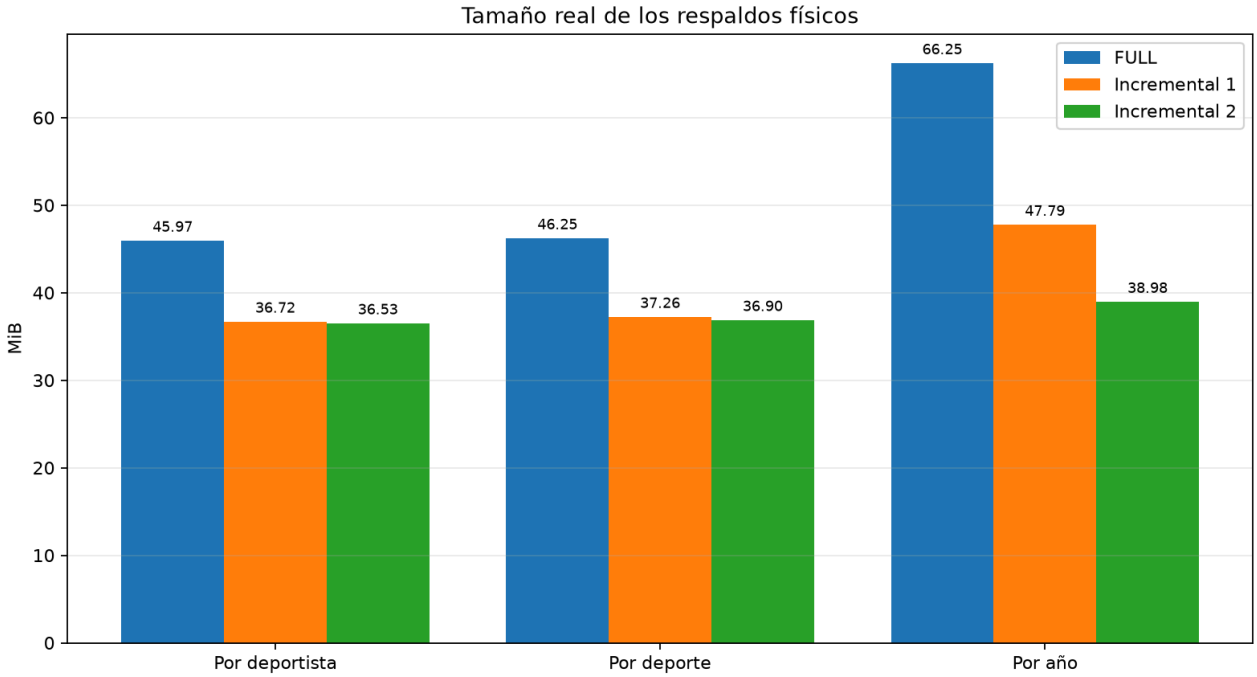
1. Tiempos de respaldo (segundos)

Ronda	Full	Incremental 1	Incremental 2
Por deportista (14 participaciones)	43.11	40.75	41.03
Por deporte (410 participaciones)	44.04	40.25	41.25
Por año (75,533 participaciones)	44.31	39.11	39.96



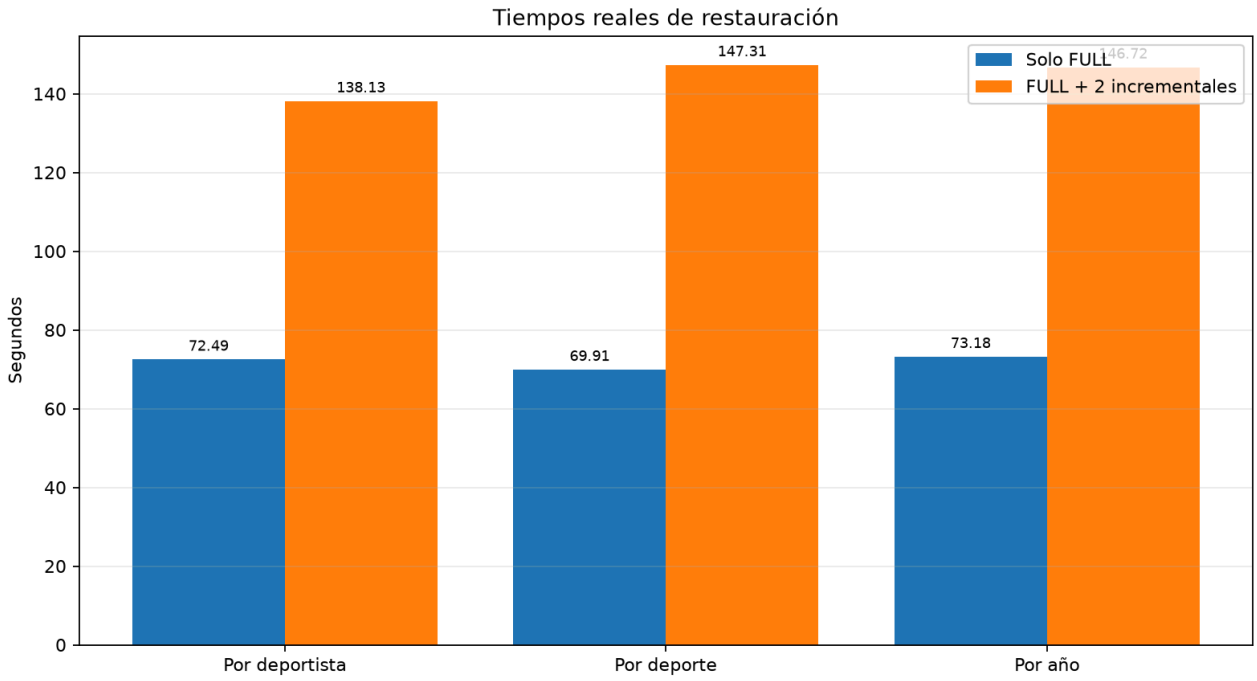
2. Tamaño de los respaldos en disco (MB)

Ronda	Full	Incremental 1	Incremental 2
Por deportista	46.0	36.7	36.5
Por deporte	46.3	37.3	36.9
Por año	66.3	47.8	39.0



3. Tiempos de restauración (segundos)

Ronda	Solo FULL	FULL + 2 incrementales (`pg_combinebackup`)	Diferencia
Por deportista	72.49	138.13	+90.6%
Por deporte	69.91	147.31	+110.7%
Por año	73.18	146.72	+100.5%



4. Hallazgos

El tiempo de respaldo NO escala con el volumen de datos en este rango.

Entre la ronda más chica (14 filas) y la más grande (75,533 filas) el tiempo de `pg_basebackup` se mantiene entre 39-45 segundos en todos los casos. La razón es que, a esta escala (decenas de MB), el costo fijo domina: esperar el checkpoint (`-c fast`), el protocolo de replicación, y el streaming de WAL (segmentos de 16 MB mínimo) pesan más que la diferencia real de datos. El tamaño en disco sí refleja el ahorro esperado: los incrementales pesan 19-41% menos que el full porque solo copian bloques modificados. Aunque la tercera carga por año agrega 31,643 participaciones, su incremental se mantuvo en 39.0 MB frente a 66.3 MB del FULL.

Restaurar con incrementales siempre fue más lento que restaurar solo el full (91%-111% más tiempo).

`pg_combinebackup` tiene que leer el full completo más cada incremental de la cadena y reconstruir un directorio de datos íntegro antes de poder arrancar Postgres; restaurar solo el full es una simple copia de directorio.

Es la contraparte del punto anterior: lo que no se paga al respaldar (tiempo) se paga al restaurar.

Restaurar solo el full backup NUNCA recupera el estado más reciente.

Se validó en las 3 rondas: tras restaurar únicamente el full, el `COUNT(*)` de `participacion` coincide exactamente con lo cargado en la carga 1 (p. ej. 14,891 de 75,533 en la ronda por año) — los datos de las cargas 2 y 3 solo aparecen al combinar el full con ambos incrementales (ver `evidencia.md`, secciones 1.3-1.4, 2.2 y 3.2).

5. Conclusión y recomendación

- **Si la prioridad es* tiempo de recuperación *(RTO bajo)** y se puede tolerar perder los cambios posteriores al último full, el respaldo **completo** es la mejor opción: restaura más rápido porque es una copia directa de un directorio ya consistente.
- **Si la prioridad es* minimizar la pérdida de datos *(RPO bajo)** — recuperar hasta el último cambio posible — es necesario el esquema **completo + incremental**, aceptando un tiempo de restauración mayor (en estas pruebas, 1.9x a 2.1x más) porque hay que recombinar la cadena.
- **Recomendación para este volumen de datos (cientos de miles de filas):** mantener la estrategia híbrida clásica — un full periódico (p. ej. diario) + incrementales frecuentes entre cada full (p. ej. cada carga) — ya que el costo de tomar un incremental es prácticamente el mismo que un full (ambos dominados por el overhead fijo medido arriba) pero el incremental pesa menos en disco, y el riesgo de una restauración más lenta solo se asume en el escenario de recuperación total, que es infrecuente.

Evidencia de ejecución — Fase 2

Capturas con fecha/hora real del sistema de una corrida completa de las 3 rondas (por año, por deporte, por deportista) el **2026-10-16**, entre las 21:24 y las 21:51. Cada ronda sigue el mismo ciclo: 3 cargas con respaldo (full tras la carga 1, incremental tras las cargas 2 y 3) → se elimina la base de datos por completo (`docker compose down -v`) → se restaura solo el full (con tiempo) → se restaura full + incrementales con `pg_combinebackup` (con tiempo) → se valida integridad con `COUNT()` y `SELECT` .

Comandos tal como se ejecutaron, para reproducir (PowerShell, desde `Fase2/`):



1. Ronda por año (París 2024 → Tokio 2020 → Río 2016)

La más grande de las 3: 75,533 participaciones acumuladas al final.

La validación final de esta ronda incluyó datos en las diez tablas:

Tabla	Filas
pais	228
poblacion	8,896
atleta	40,754
edicion	3
deporte	85
evento	1,676
equipo	5,108
club	9,384
atleta_club	15,202
participacion	75,533

La derivación enlazó 28,647 participaciones con un equipo y la validación referencial terminó con cero filas huérfanas.

1.1 Cargas 1, 2 y 3

```
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_anio\carga1_paris2024
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_anio\carga2_tokio2020
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_anio\carga3_rio2016
```

```
Resumen de las tres cargas
paso                        | participaciones | equipos | clubes
anio_1_paris2024            |          14,891 |   1,071 |  1,446
anio_2_tokio2020            |          43,890 |   3,367 |  5,863
anio_3_rio2016              |          75,533 |   5,108 |  9,384

Integridad: 0 filas huerfanas
```

Validación literal de `COUNT()`, *muestras de* `SELECT` de las diez tablas y fecha/hora después de cada carga:

```
Validación real: por_anio_carga1_paris2024

tabla          |    total
---------------+---------
pais           |      207
poblacion      |    8,832
atleta         |   11,100
edicion        |        1
deporte        |       50
evento         |      361
equipo         |    1,071
club           |    1,446
atleta_club    |    1,810
participacion  |   14,891

VALIDACION OK: 0 filas huerfanas
```

Validación real: por_anio_carga2_tokio2020

tabla	total
pais	228
poblacion	8,896
atleta	27,020
edicion	2
deporte	79
evento	1,053
equipo	3,367
club	5,863
atleta_club	8,913
participacion	43,890

VALIDACION OK: 0 filas huerfanas

Validación real: por_anio_carga3_rio2016

tabla	total
pais	228
poblacion	8,896
atleta	40,754
edicion	3
deporte	85
evento	1,676
equipo	5,108
club	9,384
atleta_club	15,202
participacion	75,533

VALIDACION OK: 0 filas huerfanas

1.2 Respaldos full + incremental

```
(
\Backup\respaldo_full.ps1 -Destino para_anio\full_carga1
\Backup\respaldo_incremental.ps1 -Destino para_anio\incr1_carga2 -BaseFrestPrise para_anio\full_carga1
\Backup\respaldo_incremental.ps1 -Destino para_anio\incr2_carga3 -BaseFrestBase
para_anio\incr1_carga2
```

```
Respaldos y restauraciones: Por año
FULL           44.31s  por_anio/full_carga1
INCREMENTAL    39.11s  por_anio/incr1_carga2
INCREMENTAL    39.96s  por_anio/incr2_carga3

RESTORE_FULL           73.18s
RESTORE_INCREMENTAL   122.73s
RESTORE_INCREMENTAL   146.72s
```

1.3 Eliminar la BD completa + restaurar solo el full

```
docker compose down -v
./restore/restaurar_full.ps1 -Backup "por_anio/full_carga1" -Puerto 5434
```

```
Restauración FULL verificada

pais           |      207
poblacion      |    8,832
atleta         |   11,100
edicion        |        1
deporte        |       50
evento         |      361
equipo         |    1,071
club           |    1,446
atleta_club    |    1,810
participacion  |   14,891

VALIDACION OK
```

Como se esperaba, solo recupera los 14,891 registros de la carga 1 — el full

backup no incluye lo cargado después.

1.4 Restaurar full + 2 incrementales (recuperación total)

```
./restore/restaurar_incremental.ps1 -Cadena
"por_anio/full_carga1,por_anio/incr1_carga2,por_anio/incr2_carga3" -Puerto 5434
```

```
Restauración incremental completa verificada

pais           |      228
poblacion      |    8,896
atleta         |   40,754
edicion        |        3
deporte        |       85
evento         |    1,676
equipo         |    5,108
club           |    9,384
atleta_club    |   15,202
participacion  |   75,533

VALIDACION OK
```

Recupera las 75,533 participaciones completas (14,891 + 28,999 + 31,643).

2. Ronda por deporte (100m mujeres: París 2024 → Tokio 2020 → Río 2016)

2.1 Cargas 1, 2 y 3

```
\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deporte\carga1_paris2024
\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deporte\carga2_tokio2020
\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deporte\carga3_rio2016
```

```
Resumen de las tres cargas
paso                  | participaciones | equipos | clubes
deporte_1_paris2024   |              91 |       0 |     13
deporte_2_tokio2020   |             232 |       0 |     58
deporte_3_rio2016     |             410 |       0 |     90

Integridad: 0 filas huerfanas
```

Validación de las diez tablas después de cada carga:

Validación real: por_deporte_carga1_paris2024

tabla	total
pais	73
poblacion	3,264
atleta	91
edicion	1
deporte	1
evento	1
equipo	0
club	13
atleta_club	13
participacion	91

VALIDACION OK: 0 filas huerfanas

Validación real: por_deporte_carga2_tokio2020

tabla	total
pais	113
poblacion	4,864
atleta	216
edicion	2
deporte	1
evento	3
equipo	0
club	58
atleta_club	60
participacion	232

VALIDACION OK: 0 filas huerfanas

Validación real: por_deporte_carga3_rio2016

tabla	total
pais	134
poblacion	5,696
atleta	311
edicion	3
deporte	1
evento	5
equipo	0
club	90
atleta_club	95
participacion	410

VALIDACION OK: 0 filas huerfanas

2.2 Respaldos y restauraciones

```
.\backup\respaldo_full.ps1 -Destino por_deporte/full_carga1
.\backup\respaldo_incremental.ps1 -Destino por_deporte/incr1_carga2 -WalTestBase
por_deporte/full_carga1
.\backup\respaldo_incremental.ps1 -Destino por_deporte/incr2_carga3 -WalTestBase
por_deporte/incr1_carga2

docker compose down -v
.\restore\restaurar_full.ps1 -BackupDir por_deporte/full_carga1 -Puerto 5434
.\restore\restaurar_incremental.ps1 -Fuente
por_deporte/full_carga1,por_deporte/incr1_carga2,por_deporte/incr2_carga3 -Puerto 5434
```

Respaldos y restauraciones: Por deporte

FULL	44.04s	por_deporte/full_carga1
INCREMENTAL	40.25s	por_deporte/incr1_carga2
INCREMENTAL	41.25s	por_deporte/incr2_carga3

RESTORE_FULL	69.91s
RESTORE_INCREMENTAL	147.31s

Full-only: 91 (solo carga 1). Full + incrementales: 410 = 91 + 141 + 178,

desglosado correctamente por año (2016: 178, 2020: 141, 2024: 91).

3. Ronda por deportista (Usain Bolt: Atenas 2004 → Pekín 2008 → Londres 2012)

3.1 Cargas 1, 2 y 3

```
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deportista\carga1_atenas2004
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deportista\carga2_pekin2008
.\carga\cargar_paso.ps1 -CarpetaCSV .\carga\salida\conjuntos\por_deportista\carga3_londres2012
```

```
Resumen de las tres cargas
paso                        | participaciones | equipos | clubes
deportista_1_atenas2004     |               2 |       0 |      1
deportista_2_pekin2008      |               8 |       0 |      1
deportista_3_londres2012    |              14 |       0 |      1

Integridad: 0 filas huerfanas
```

Validación de las diez tablas después de cada carga:

```
Validación real: por_deportista_carga1_atenas2004

tabla           |    total
----------------+---------
pais            |        2
poblacion       |      128
atleta          |        1
edicion         |        1
deporte         |        1
evento          |        2
equipo          |        0
club            |        1
atleta_club     |        1
participacion   |        2

VALIDACION OK: 0 filas huerfanas
```

```
Validación real: por_deportista_carga2_pekin2008

tabla           |    total
----------------+---------
pais            |        3
poblacion       |      192
atleta          |        1
edicion         |        2
deporte         |        1
evento          |        8
equipo          |        0
club            |        1
atleta_club     |        1
participacion   |        8

VALIDACION OK: 0 filas huerfanas
```

```
Validación real: por_deportista_carga3_londres2012

tabla           |    total
----------------+---------
pais            |        4
poblacion       |      256
atleta          |        1
edicion         |        3
deporte         |        1
evento          |       14
equipo          |        0
club            |        1
atleta_club     |        1
participacion   |       14

VALIDACION OK: 0 filas huerfanas
```

Se ven sus 2 medallas de oro individuales en Pekín 2008 (100m, 200m) — el relevo 4x100m de esa edición aparece sin medalla (`AC`) porque fue anulado en 2017 al reanalizarse una muestra de un compañero de equipo, y así quedó en la fuente de datos — y las 3 medallas de oro de Londres 2012 (100m, 200m, relevo 4x100m, sin anular).

3.2 Respaldos y restauraciones

```
backup_respaldo_antes_de_cargar_por_deportista_carga2
```

```
./backup/respaldo_incremental.ps1 -Destino por_deportista/incr1_carga2 -BaseManifest
por_deportista/full_carga1
./backup/respaldo_incremental.ps1 -Destino por_deportista/incr2_carga3 -BaseManifest
por_deportista/incr1_carga2

docker compose down -v
./restore/restaurar_full.ps1 -BackupDir por_deportista/full_carga1 -Puerto 5433
./restore/restaurar_incremental.ps1 -Cadena
por_deportista/full_carga1,por_deportista/incr1_carga2,por_deportista/incr2_carga3 -Puerto 5433
```

```
Respaldos y restauraciones: Por deportista
FULL            43.11s  por_deportista/full_carga1
INCREMENTAL     40.75s  por_deportista/incr1_carga2
INCREMENTAL     41.03s  por_deportista/incr2_carga3

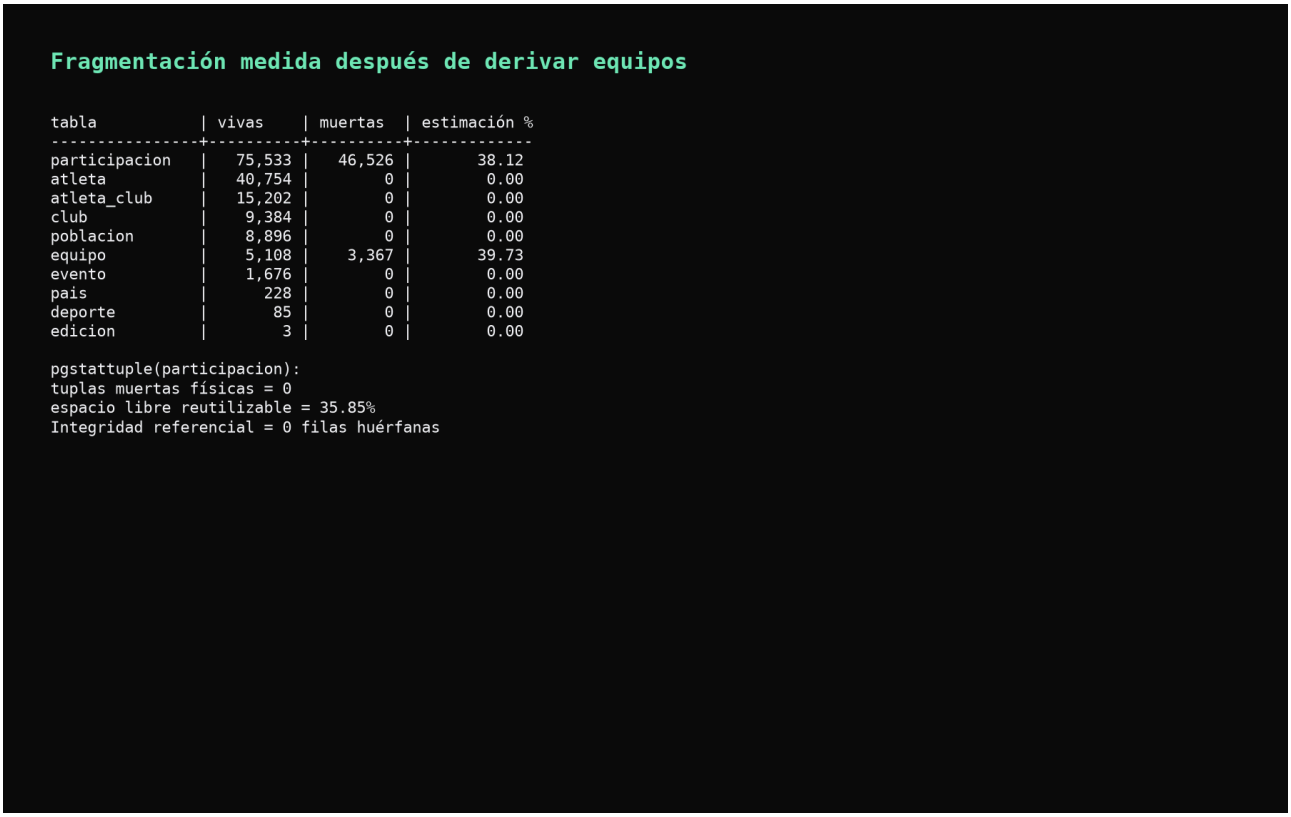
RESTORE_FULL            72.49s
RESTORE_INCREMENTAL    138.13s
```

Full-only: 2 filas (solo Atenas 2004). Full + incrementales: 14 filas

completas, con las medallas de 2008 y 2012 presentes.

4. Fragmentación de tablas

```
docker cp sql/verificar_fragmentacion.sql fase2-pgs:/verificar_fragmentacion.sql
docker exec fase2-pgs psql -U postgres -d olimpiadas_fase2 -f /verificar_fragmentacion.sql
```



La carga masiva usa `COPY`; adicionalmente, la derivación de equipos ejecuta `UPDATE` y `DELETE`. Por MVCC pueden existir tuplas muertas hasta que `autovacuum` las limpie. La condición obligatoria de integridad se comprobó independientemente: cero filas huérfanas en las diez tablas. Ver la explicación en `informe_tecnico.md` sección 5. En la corrida final, `pg_stat_user_tables` todavía estimó 38.12% para `participacion` y 39.73% para `equipo`, mientras `pgstattuple` confirmó físicamente 0 tuplas muertas en `participacion` después de `autovacuum` (35.85% de espacio reutilizable).

5. Logs completos

Los tiempos exactos de cada operación (con timestamp ISO) quedaron registrados en `../logs/backups.csv` y `../logs/restores.csv`, resumidos en `analisis_comparativo.md`.

6. Revalidación funcional final

El **2026-10-16 entre 21:24 y 21:51** se repitieron de forma independiente

las seis restauraciones en el contenedor aislado `fase2-validacion`, sin

alterar `fase2-pg`. Los resultados fueron:

Ronda	Conteo tras FULL	Conteo tras FULL + incrementales	Integridad
Por deportista	2	14	0 huérfanos
Por deporte	91	410	0 huérfanos
Por año	14,891	75,533	0 huérfanos

La salida y los tiempos de esta corrida están en

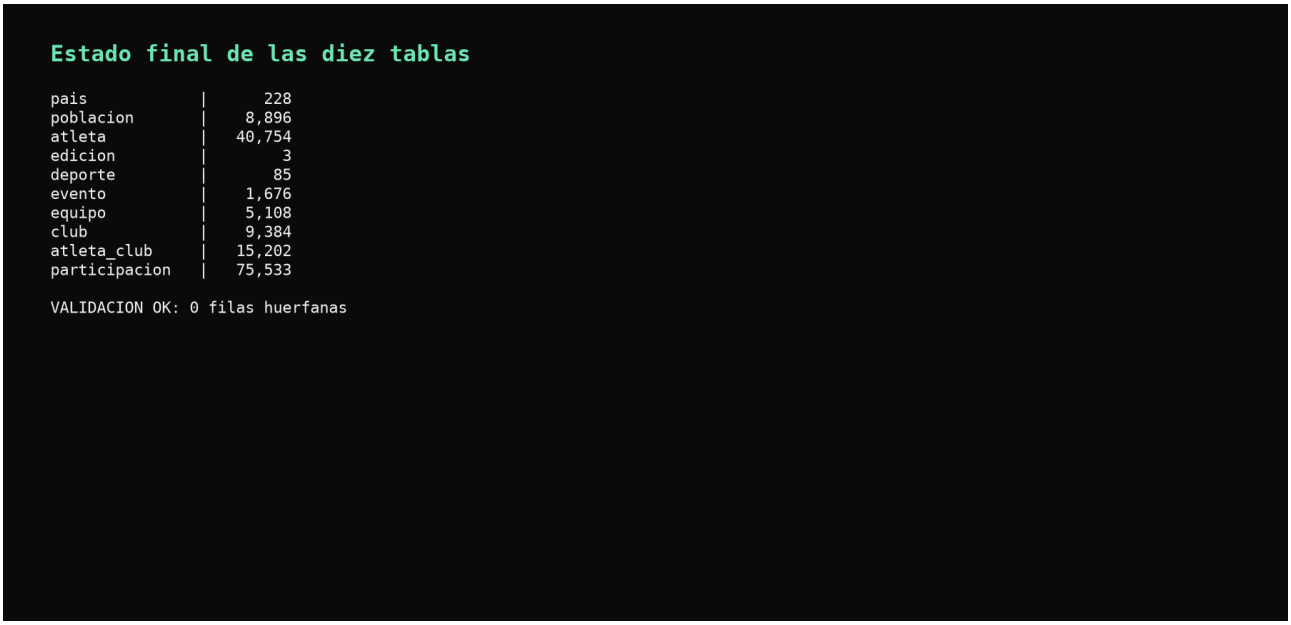
`../logs/restores.csv` y en

`../logs/validacion_restore_*.txt`. La base principal también se validó con

`../sql/validar_integridad.sql`: registra la

fecha/hora del servidor, ejecuta `COUNT()`, *muestras de* `SELECT` sobre las

diez tablas y aborta si encuentra relaciones huérfanas.



Capturas equivalentes de cada restauración completa:

- Por año
- Por deporte
- Por deportista